

---

# [ 머니새니 ]

## 소프트웨어 설계서

---

2026년 5월 23일

|       |                                                                                                             |
|-------|-------------------------------------------------------------------------------------------------------------|
| 문서 번호 | [이나경] 소프트웨어설계서_260601_Doc-004                                                                               |
| 소 속   | 한국항공대학교 소프트웨어학과                                                                                             |
| 작 성 자 | 이나경                                                                                                         |
| 학 번   | 2024125039                                                                                                  |
| 깃 허 브 | <a href="https://github.com/Niicka/Software-Engineering">https://github.com/Niicka/Software-Engineering</a> |

---

## 제/개정 이력

| 버전   | 날짜         | 작성자 | 변경 내용 |
|------|------------|-----|-------|
| v1.0 | 2026.06.01 | 이나경 | 최초 작성 |

## 목 차

|                                               |    |
|-----------------------------------------------|----|
| 1. 서론 .....                                   | 1  |
| 1.1 목적 및 범위 .....                             | 1  |
| 1.2 용어 정의 .....                               | 1  |
| 1.3 참조 문서 .....                               | 1  |
| 2. 소프트웨어 아키텍처 .....                           | 2  |
| 2.1 정적 구조 .....                               | 2  |
| 2.2 동적 구조 .....                               | 3  |
| 3. 모듈/패키지 설계 .....                            | 4  |
| 3.1 SDD-P-001 : Auth (사용자 인증 패키지) .....       | 4  |
| 3.2 SDD-P-002 : Consumption (소비 관리 패키지) ..... | 6  |
| 3.3 SDD-P-003 : Challenge (챌린지/보상 패키지) .....  | 8  |
| 3.4 SDD-P-004 : AI (AI 분석 패키지) .....          | 10 |
| 3.5 SDD-P-005 : Community (커뮤니티 패키지) .....    | 12 |
| 4. 인터페이스 설계 .....                             | 13 |
| 4.1 외부 시스템 인터페이스 .....                        | 13 |
| 4.2 사용자 인터페이스 .....                           | 13 |
| 5. 데이터 설계 .....                               | 14 |
| 6. 구현 기술 설계 .....                             | 16 |
| 7. 요구사항 추적표 .....                             | 17 |
| 8. 부록 .....                                   | 18 |

# 1. 서론

## 1.1 목적 및 범위

본 문서는 '머니새니(Money Sani)' 앱 개발 프로젝트의 소프트웨어 컴포넌트에 대한 상세 설계 사항을 기록하고 문서화하기 위해 작성된 소프트웨어 설계서이다. 요구사항 정의서(Doc-003)에서 도출된 기능 및 비기능 요구사항을 기반으로, 시스템을 구성하는 아키텍처, 모듈/패키지 구조, 인터페이스, 데이터, 구현 기술에 대한 설계 결정 사항을 명확히 기술한다.

본 문서의 범위는 다음과 같다.

- React Native(Frontend), Node.js/Spring Boot(Backend), Python/Flask(AI 서버)를 포함한 전체 시스템 아키텍처 설계
- 사용자 인증, 소비 관리, 챌린지/보상, AI 분석, 커뮤니티 등 5 개 핵심 패키지의 클래스 및 메소드 설계
- 외부 시스템(카드사 Open API, FCM, 기프트콘 파트너 API) 인터페이스 설계
- 데이터베이스 스키마(ERD 기반 주요 엔티티) 설계

## 1.2 용어 정의

| 용어/약어         | 정의                                                         |
|---------------|------------------------------------------------------------|
| SDD           | Software Design Document. 소프트웨어 설계서.                       |
| 패키지 (Package) | 관련 클래스들의 집합. 본 문서에서는 기능 도메인 단위로 구분.                        |
| 클래스 (Class)   | 객체지향 설계의 기본 단위. 속성(Attribute)과 메소드(Method)로 구성.            |
| REST API      | HTTP 기반의 인터페이스. Frontend와 Backend 간 통신에 사용.                |
| JWT           | JSON Web Token. 사용자 인증 및 세션 관리에 사용되는 토큰 기반 인증 방식.          |
| FCM           | Firebase Cloud Messaging. 푸시 알림 전송 서비스.                    |
| ERD           | Entity Relationship Diagram. 데이터베이스 엔티티 간의 관계를 나타내는 다이어그램. |
| ORM           | Object-Relational Mapping. 객체와 관계형 DB를 자동 매핑하는 기술.         |

## 1.3 참조 문서

| 문서명           | 문서 번호   | 작성일        |
|---------------|---------|------------|
| [이나경] 시스템 정의서 | Doc-001 | 2026.03.16 |

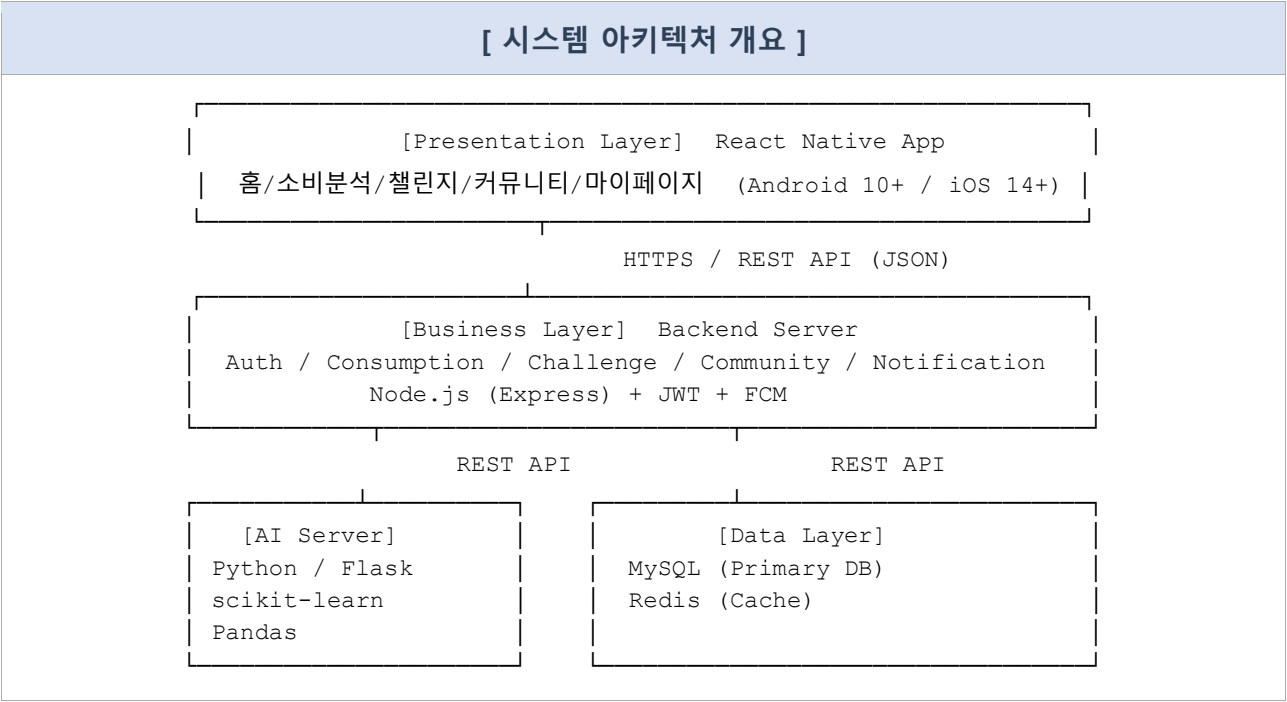
---

|                   |         |            |
|-------------------|---------|------------|
| [이나경] 프로젝트 관리 계획서 | Doc-002 | 2026.04.09 |
| [이나경] 요구사항 정의서    | Doc-003 | 2026.05.13 |

## 2. 소프트웨어 아키텍처

### 2.1 정적 구조

머니새니 시스템은 3-Tier 아키텍처를 기반으로 설계되며, Frontend(React Native 앱), Backend(REST API 서버), AI 서버(Python/Flask), Database(MySQL)로 구성된다.



| 계층           | 컴포넌트             | 설명                                                                     |
|--------------|------------------|------------------------------------------------------------------------|
| Presentation | React Native App | 모바일 앱 UI. 5개 주요 화면(홈, 소비분석, 챌린지, 커뮤니티, 마이페이지)으로 구성.                    |
| Business     | Backend Server   | REST API 제공. Auth, Consumption, Challenge, Community, Notification 모듈. |
| AI           | AI Server        | Python/Flask 기반 충동소비 탐지 및 개인화 절약 팁 추천 엔진.                              |
| Data         | MySQL / Redis    | 관계형 DB(MySQL)와 캐시(Redis)로 구성.<br>ORM(Sequelize/Hibernate) 사용.          |

### 2.2 동적 구조

아래는 머니새니의 핵심 시나리오별 주요 시퀀스 흐름을 나타낸다.

---

### [ 시나리오 1: 소비 내역 입력 및 AI 분석 흐름 ]

사용자(App) → Backend: POST /v1/consumptions (소비 내역 입력)  
Backend → Database: 소비 내역 저장 (INSERT)  
Backend → AI Server: POST /ai/detect (소비 데이터 전달)  
AI Server → Backend: 총동소비 탐지 결과 + 절약 팁 반환  
Backend → FCM: 총동소비 탐지 시 푸시 알림 발송  
Backend → App: 응답(소비 저장 결과 + AI 팁) 반환

### [ 시나리오 2: 챌린지 참여 및 보상 지급 흐름 ]

사용자(App) → Backend: POST /v1/challenges/{id}/join (챌린지 참여)  
Backend → Database: 챌린지 참여 정보 저장  
Scheduler → Backend: 챌린지 종료 시 성공/실패 자동 판정  
Backend → Database: 포인트/배지 지급 처리 (트랜잭션)  
Backend → FCM: 챌린지 결과 푸시 알림 발송

---

### 3. 모듈/패키지 설계

---

#### 3.1 SDD-P-001 : Auth (사용자 인증 패키지)

##### 3.1.1 패키지 설명

| 항목      | 내용                                                                        |
|---------|---------------------------------------------------------------------------|
| 패키지 ID  | SDD-P-001                                                                 |
| 패키지명    | Auth                                                                      |
| 담당 레이어  | Backend Server                                                            |
| 설명      | 사용자 회원가입, 로그인, 소셜 로그인(OAuth2.0), JWT 토큰 발급/검증, 비밀번호 재설정, 프로필 관리 기능을 담당한다. |
| 관련 요구사항 | FR-001 ~ FR-006 (사용자 인증 및 계정 관리)                                          |

##### 3.1.2 구성 클래스 설계

(1) CM-001 : UserAccount

| 항목     | 이름                    | 설명                             |
|--------|-----------------------|--------------------------------|
| 클래스 ID | CM-001                | UserAccount                    |
| 속성     | userId : String       | 사용자 고유 식별자 (UUID)              |
|        | email : String        | 로그인 이메일 (unique)               |
|        | passwordHash : String | bcrypt 해시된 비밀번호                |
|        | nickname : String     | 사용자 닉네임                        |
|        | provider : Enum       | 인증 방식 (LOCAL / KAKAO / GOOGLE) |
|        | createdAt : DateTime  | 계정 생성 일시                       |
| 메소드    | register()            | M-001 참조                       |
|        | login()               | M-002 참조                       |
|        | updateProfile()       | M-003 참조                       |
|        | withdraw()            | M-004 참조                       |

(2) CM-002 : JwtTokenManager

| 항목     | 이름                       | 설명                                  |
|--------|--------------------------|-------------------------------------|
| 클래스 ID | CM-002                   | JwtTokenManager                     |
| 속성     | secretKey : String       | JWT 서명 비밀키 (환경변수)                   |
|        | accessTokenExpiry : Int  | Access Token 유효기간 (3600초 = 1시간)     |
|        | refreshTokenExpiry : Int | Refresh Token 유효기간 (2592000초 = 30일) |
| 메소드    | generateToken()          | M-005 참조                            |
|        | validateToken()          | M-006 참조                            |

### 3.1.3 패키지 행위 (시퀀스)

| [ 로그인 시퀀스 ]     |                                                          |
|-----------------|----------------------------------------------------------|
| App             | → AuthController: POST /v1/auth/login {email, password}  |
| AuthController  | → UserAccount: login(email, password) 호출                 |
| UserAccount     | → DB: 이메일로 사용자 조회 SELECT * FROM users WHERE email=?      |
| UserAccount     | → UserAccount: bcrypt.compare(password, passwordHash) 검증 |
| UserAccount     | → JwtTokenManager: generateToken(userId) 호출              |
| JwtTokenManager | → AuthController: {accessToken, refreshToken} 반환         |
| AuthController  | → App: 200 OK {accessToken, refreshToken, user}          |

### 3.1.4 멤버함수(메소드) 설계

(1) M-001 : register()

| 항목     | 내용                                                            |
|--------|---------------------------------------------------------------|
| 메소드 ID | M-001                                                         |
| 메소드명   | register(email, password, nickname)                           |
| 설명     | 신규 사용자 회원가입. 이메일 중복 확인 후 비밀번호를 bcrypt 해시하여 DB에 저장한다.          |
| 입력     | email: String, password: String, nickname: String             |
| 출력     | 201 Created {userId, email, nickname} / 409 Conflict (이메일 중복) |
| 예외처리   | 이메일 형식 오류(400), 비밀번호 정책 위반(400), DB 오류(500)                   |

(2) M-002 : login()

| 항목 | 내용 |
|----|----|
|----|----|

|        |                                                             |
|--------|-------------------------------------------------------------|
| 메소드 ID | M-002                                                       |
| 메소드명   | login(email, password)                                      |
| 설명     | 이메일/비밀번호 검증 후 JWT Access/Refresh Token을 발급한다.               |
| 입력     | email: String, password: String                             |
| 출력     | 200 OK {accessToken, refreshToken, user} / 401 Unauthorized |
| 예외처리   | 계정 없음(404), 비밀번호 불일치(401), 계정 잠금(423)                       |

(3) M-005 : generateToken()

| 항목     | 내용                                                               |
|--------|------------------------------------------------------------------|
| 메소드 ID | M-005                                                            |
| 메소드명   | generateToken(userId)                                            |
| 설명     | userId를 페이로드로 하는 JWT Access Token(1h)과 Refresh Token(30d)을 생성한다. |
| 입력     | userId: String                                                   |
| 출력     | {accessToken: String, refreshToken: String}                      |

## 3.2 SDD-P-002 : Consumption (소비 관리 패키지)

### 3.2.1 패키지 설명

| 항목      | 내용                                                                |
|---------|-------------------------------------------------------------------|
| 패키지 ID  | SDD-P-002                                                         |
| 패키지명    | Consumption                                                       |
| 담당 레이어  | Backend Server                                                    |
| 설명      | 소비 내역 수동 입력/자동 연동, 카테고리 분류, 주간/월간 리포트 생성, 예산 설정 및 초과 알림 기능을 담당한다. |
| 관련 요구사항 | FR-010 ~ FR-024 (소비 리포트, 예산 설정)                                   |

### 3.2.2 구성 클래스 설계

(1) CM-003 : ConsumptionRecord

| 항목     | 이름                    | 설명                                                              |
|--------|-----------------------|-----------------------------------------------------------------|
| 클래스 ID | CM-003                | ConsumptionRecord                                               |
| 속성     | recordId : String     | 소비 내역 고유 ID                                                     |
|        | userId : String       | 소비한 사용자 ID (FK)                                                 |
|        | amount : Long         | 소비 금액 (원 단위)                                                    |
|        | category : Enum       | 카테고리<br>(FOOD/SHOPPING/CAFE/TRANSPORT/SUBSCRIPTION/LEISURE/ETC) |
|        | memo : String         | 메모 (선택)                                                         |
|        | consumedAt : DateTime | 소비 발생 일시                                                        |
| 메소드    | createRecord()        | M-007 참조                                                        |
|        | getCategoryStats()    | M-008 참조                                                        |
|        | getWeeklyReport()     | M-009 참조                                                        |
|        | getMonthlyReport()    | M-010 참조                                                        |

(2) CM-004 : BudgetManager

| 항목     | 이름                 | 설명                           |
|--------|--------------------|------------------------------|
| 클래스 ID | CM-004             | BudgetManager                |
| 속성     | budgetId : String  | 예산 고유 ID                     |
|        | userId : String    | 예산 소유자 ID (FK)               |
|        | category : Enum    | 예산 카테고리 (전체/개별)              |
|        | limitAmount : Long | 예산 한도 금액                     |
|        | period : Enum      | 예산 기간 (DAILY/WEEKLY/MONTHLY) |
| 메소드    | setBudget()        | M-011 참조                     |
|        | checkOverBudget()  | M-012 참조                     |

### 3.2.3 패키지 행위 (시퀀스)

#### [ 소비 입력 및 예산 초과 알림 시퀀스 ]

App → ConsumptionController: POST /v1/consumptions  
 ConsumptionController → ConsumptionRecord: createRecord() 호출  
 ConsumptionRecord → DB: INSERT INTO consumption\_records  
 ConsumptionController → BudgetManager: checkOverBudget(userId, category) 호출  
 BudgetManager → DB: 해당 기간 카테고리 합계 조회  
 BudgetManager: 80% 초과 → WARN / 100% 초과 → EXCEED 반환  
 ConsumptionController → FCM: 초과 시 푸시 알림 발송

### 3.2.4 멤버함수(메소드) 설계

(1) M-007 : createRecord()

| 항목     | 내용                                                                               |
|--------|----------------------------------------------------------------------------------|
| 메소드 ID | M-007                                                                            |
| 메소드명   | createRecord(userId, amount, category, memo, consumedAt)                         |
| 설명     | 소비 내역을 DB에 저장하고, 저장 후 예산 초과 여부를 확인한다.                                            |
| 입력     | userId: String, amount: Long, category: Enum, memo: String, consumedAt: DateTime |
| 출력     | 201 Created {recordId, amount, category, consumedAt}                             |
| 예외처리   | 금액 음수(400), 카테고리 오류(400), 인증 실패(401)                                             |

---

(2) M-009 : getWeeklyReport()

| 항목     | 내용                                                                                   |
|--------|--------------------------------------------------------------------------------------|
| 메소드 ID | M-009                                                                                |
| 메소드명   | getWeeklyReport(userId, startDate)                                                   |
| 설명     | 지정된 주간의 카테고리별 지출 통계를 집계하여 그래프 데이터로 반환한다.                                             |
| 입력     | userId: String, startDate: Date (주 시작일)                                              |
| 출력     | {totalAmount, categories: [{category, amount, ratio}], dailyTrend: [{date, amount}]} |

### 3.3 SDD-P-003 : Challenge (챌린지/보상 패키지)

#### 3.3.1 패키지 설명

| 항목      | 내용                                                                    |
|---------|-----------------------------------------------------------------------|
| 패키지 ID  | SDD-P-003                                                             |
| 패키지명    | Challenge                                                             |
| 담당 레이어  | Backend Server                                                        |
| 설명      | 절약 챌린지 목록 제공, 참여 관리, 성공/실패 자동 판정, 포인트/배지 보상 지급, 공동 챌린지 및 랭킹 기능을 담당한다. |
| 관련 요구사항 | FR-030 ~ FR-044 (절약 챌린지, 보상 및 인센티브)                                   |

#### 3.3.2 구성 클래스 설계

##### (1) CM-005 : Challenge

| 항목     | 이름                   | 설명                       |
|--------|----------------------|--------------------------|
| 클래스 ID | CM-005               | Challenge                |
| 속성     | challengeId : String | 챌린지 고유 ID                |
|        | title : String       | 챌린지 제목 (예: '하루 커피값 아끼기') |
|        | description : String | 챌린지 상세 설명                |
|        | difficulty : Enum    | 난이도 (EASY/MEDIUM/HARD)   |
|        | rewardPoints : Int   | 성공 시 지급 포인트              |
|        | durationDays : Int   | 챌린지 기간 (일 단위)            |
| 메소드    | joinChallenge()      | M-013 참조                 |
|        | evaluateResult()     | M-014 참조                 |

##### (2) CM-006 : RewardSystem

| 항목     | 이름                   | 설명           |
|--------|----------------------|--------------|
| 클래스 ID | CM-006               | RewardSystem |
| 속성     | pointBalance : Long  | 현재 보유 포인트    |
|        | badges : List<Badge> | 보유 배지 목록     |

|            |                |               |
|------------|----------------|---------------|
|            | level : Int    | 사용자 레벨 (1~10) |
| <b>메소드</b> | grantPoints()  | M-015 참조      |
|            | grantBadge()   | M-016 참조      |
|            | redeemPoints() | M-017 참조      |

### 3.3.3 멤버함수(메소드) 설계

(1) M-013 : joinChallenge()

| 항목            | 내용                                                              |
|---------------|-----------------------------------------------------------------|
| <b>메소드 ID</b> | M-013                                                           |
| <b>메소드명</b>   | joinChallenge(userId, challengeId)                              |
| <b>설명</b>     | 사용자가 챌린지에 참여 신청한다. 이미 참여 중인 경우 중복 참여를 방지한다.                     |
| <b>입력</b>     | userId: String, challengeId: String                             |
| <b>출력</b>     | 201 Created {participationId, startDate, endDate} / 409 (중복 참여) |

(2) M-014 : evaluateResult()

| 항목            | 내용                                                                                      |
|---------------|-----------------------------------------------------------------------------------------|
| <b>메소드 ID</b> | M-014                                                                                   |
| <b>메소드명</b>   | evaluateResult(participationId)                                                         |
| <b>설명</b>     | 챌린지 기간 만료 시 Scheduler가 호출. 조건(해당 카테고리 소비 내역) 충족 여부를 판정하여 성공/실패 처리하고, 성공 시 보상 지급을 트리거한다. |
| <b>입력</b>     | participationId: String                                                                 |
| <b>출력</b>     | {result: SUCCESS FAIL, pointsAwarded: Int, badgeAwarded: String null}                   |

### 3.4 SDD-P-004 : AI (AI 분석 패키지)

#### 3.4.1 패키지 설명

| 항목      | 내용                                                                  |
|---------|---------------------------------------------------------------------|
| 패키지 ID  | SDD-P-004                                                           |
| 패키지명    | AI                                                                  |
| 담당 레이어  | AI Server (Python/Flask)                                            |
| 설명      | 소비 패턴 분석, 충동소비 탐지, 개인화 절약 팁 제안, 대체 소비 제안, 월간 AI 분석 리포트 생성 기능을 담당한다. |
| 관련 요구사항 | FR-050 ~ FR-053 (AI 기반 충동소비 탐지 및 대안 제시)                             |

#### 3.4.2 구성 클래스 설계

(1) CM-007 : ImpulseDetector

| 항목     | 이름                      | 설명                             |
|--------|-------------------------|--------------------------------|
| 클래스 ID | CM-007                  | ImpulseDetector                |
| 속성     | model : sklearn.Model   | 학습된 이상 탐지 모델 (IsolationForest) |
|        | threshold : Float       | 충동소비 판단 임계값 (기본값: 0.7)         |
|        | ruleEngine : RuleEngine | MVP용 규칙 기반 탐지 엔진 (ML 미달 시 대체)  |
| 메소드    | detect()                | M-018 참조                       |
|        | getTips()               | M-019 참조                       |
|        | generateMonthlyReport() | M-020 참조                       |

#### 3.4.3 멤버함수(메소드) 설계

(1) M-018 : detect()

| 항목     | 내용                                                                                           |
|--------|----------------------------------------------------------------------------------------------|
| 메소드 ID | M-018                                                                                        |
| 메소드명   | detect(consumptionData)                                                                      |
| 설명     | 소비 내역 데이터를 분석하여 충동소비 여부를 탐지한다. ML 모델 우선 적용, 성능 미달 시 규칙 기반(자정~새벽 고액 결제, 평균 대비 3배 이상 금액 등) 적용. |

|    |                                                                          |
|----|--------------------------------------------------------------------------|
| 입력 | consumptionData: {userId, amount, category, consumedAt, recentHistory[]} |
| 출력 | {isImpulse: Boolean, confidence: Float, reason: String}                  |

(2) M-019 : getTips()

| 항목     | 내용                                                                                |
|--------|-----------------------------------------------------------------------------------|
| 메소드 ID | M-019                                                                             |
| 메소드명   | getTips(userId, monthlyStats)                                                     |
| 설명     | 사용자의 월별 소비 통계를 분석하여 개인화된 절약 팁 3~5개를 생성한다. 가장 과소비 카테고리를 우선 타겟팅한다.                  |
| 입력     | userId: String, monthlyStats: {category: String, amount: Long, avgAmount: Long}[] |
| 출력     | {tips: [{category, message, suggestedAction, relatedChallengeId}]}                |

### 3.5 SDD-P-005 : Community (커뮤니티 패키지)

#### 3.5.1 패키지 설명

| 항목      | 내용                                                          |
|---------|-------------------------------------------------------------|
| 패키지 ID  | SDD-P-005                                                   |
| 패키지명    | Community                                                   |
| 담당 레이어  | Backend Server                                              |
| 설명      | 절약 노하우 게시글 작성/조회, 공감(좋아요), 댓글/대댓글, 신고/차단, 커뮤니티 알림 기능을 담당한다. |
| 관련 요구사항 | FR-060 ~ FR-065 (절약 커뮤니티)                                   |

#### 3.5.2 구성 클래스 설계

(1) CM-008 : Post

| 항목     | 이름                 | 설명                               |
|--------|--------------------|----------------------------------|
| 클래스 ID | CM-008             | Post                             |
| 속성     | postId : String    | 게시글 고유 ID                        |
|        | authorId : String  | 작성자 ID (FK)                      |
|        | content : String   | 게시글 내용                           |
|        | likeCount : Int    | 좋아요 수                            |
|        | commentCount : Int | 댓글 수                             |
|        | status : Enum      | 상태 (ACTIVE / REPORTED / DELETED) |
| 메소드    | createPost()       | M-021 참조                         |
|        | likePost()         | M-022 참조                         |
|        | reportPost()       | M-023 참조                         |

#### 3.5.3 멤버함수(메소드) 설계

(1) M-021 : createPost()

| 항목     | 내용                                     |
|--------|----------------------------------------|
| 메소드 ID | M-021                                  |
| 메소드명   | createPost(userId, content, imageUrls) |

---

|      |                                                           |
|------|-----------------------------------------------------------|
| 설명   | 절약 노하우 게시글을 작성한다. 금융 민감 정보(계좌번호 등) 포함 여부 필터링 후 저장한다.      |
| 입력   | userId: String, content: String, imageUrls: String[] (선택) |
| 출력   | 201 Created {postId, createdAt}                           |
| 예외처리 | 콘텐츠 정책 위반(400), 인증 실패(401)                                |

## 4. 인터페이스 설계

### 4.1 외부 시스템 인터페이스

| 인터페이스 ID | 외부 시스템                         | 프로토콜             | 설명                                                           |
|----------|--------------------------------|------------------|--------------------------------------------------------------|
| IF-001   | 카드사 Open API (오픈뱅킹)            | HTTPS / OAuth2.0 | 사용자 동의 기반 카드 소비 내역 자동 수집. API 정책 변경 대비 Mock 데이터 fallback 지원. |
| IF-002   | FCM (Firebase Cloud Messaging) | HTTPS / HTTP/2   | 예산 초과, 챌린지 결과, AI 알림, 커뮤니티 알림 등 푸시 알림 전송.                    |
| IF-003   | 기프트콘 파트너 API                   | HTTPS / REST     | 포인트-기프트콘 교환 처리. MVP에서는 Mock 처리, 파트너 계약 후 실 연동.               |
| IF-004   | AI 서버 (내부)                     | HTTPS / REST     | Backend → AI 서버 간 소비 데이터 전달 및 분석 결과 수신.                      |
| IF-005   | Kakao / Google OAuth2.0        | HTTPS / OAuth2.0 | 소셜 로그인 인증. 토큰 검증 후 내부 JWT 발급.                                |

### 4.2 사용자 인터페이스

머니새니 앱은 하단 탭 네비게이션(홈, 소비분석, 챌린지, 커뮤니티, 마이페이지) 구조로 구성되며, 주요 화면은 다음과 같다.

| 화면 ID  | 화면명      | 주요 UI 요소 및 설명                                             |
|--------|----------|-----------------------------------------------------------|
| UI-001 | 홈 (대시보드) | 오늘의 소비 현황, 예산 달성률 프로그레스 바, 진행 중 챌린지 카드, AI 절약 팁 배너        |
| UI-002 | 소비 분석    | 카테고리별 파이 차트, 기간별 바 차트, 주간/월간 리포트 탭, 소비 내역 리스트 (수동 입력 FAB) |
| UI-003 | 챌린지      | 챌린지 목록 카드 (난이도/기간/보상 표시), 참여 중 챌린지 진행바, 랭킹 탭, 히스토리 탭      |
| UI-004 | 커뮤니티     | 게시글 피드 (최신순/인기순), 게시글 작성 버튼, 좋아요/댓글 인터랙션                  |
| UI-005 | 마이페이지    | 프로필 정보, 포인트 잔액, 배지 컬렉션, 레벨 표시, 예산 설정, 알림 설정, 다크모드 토글      |

---

|        |          |                                    |
|--------|----------|------------------------------------|
| UI-006 | 소비 내역 입력 | 날짜 선택, 금액 입력, 카테고리 선택 (아이콘), 메모 입력 |
|--------|----------|------------------------------------|

---

## 5. 데이터 설계

---

주요 엔티티의 데이터 설계는 다음과 같다.

### (1) DD-01 : users (사용자)

| 컬럼명            | 타입           | 제약조건             | 설명                         |
|----------------|--------------|------------------|----------------------------|
| user_id        | VARCHAR(36)  | PK, NOT NULL     | 사용자 UUID                   |
| email          | VARCHAR(100) | UNIQUE, NOT NULL | 로그인 이메일                    |
| password_hash  | VARCHAR(255) | NULL 허용          | bcrypt 해시 (소셜 로그인 시 NULL)  |
| nickname       | VARCHAR(50)  | NOT NULL         | 닉네임                        |
| provider       | ENUM         | NOT NULL         | LOCAL/KAKAO/GOOGLE         |
| monthly_income | BIGINT       | NULL 허용          | 월 소득 (선택)                  |
| created_at     | DATETIME     | NOT NULL         | 계정 생성 일시                   |
| status         | ENUM         | NOT NULL         | ACTIVE/SUSPENDED/WITHDRAWN |

### (2) DD-02 : consumption\_records (소비 내역)

| 컬럼명       | 타입          | 제약 조건        | 설명                                                    |
|-----------|-------------|--------------|-------------------------------------------------------|
| record_id | VARCHAR(36) | PK, NOT NULL | 소비 내역 UUID                                            |
| user_id   | VARCHAR(36) | FK → users   | 소비한 사용자 ID                                            |
| amount    | BIGINT      | NOT NULL     | 소비 금액 (원)                                             |
| category  | ENUM        | NOT NULL     | FOOD/SHOPPING/CAFE/TRANSPORT/SUBSCRIPTION/LEISURE/ETC |

|             |              |          |                                  |
|-------------|--------------|----------|----------------------------------|
|             |              | L        |                                  |
| memo        | VARCHAR(200) | NULL 허용  | 메모                               |
| consumed_at | DATETIME     | NOT NULL | 소비 발생 일시                         |
| source      | ENUM         | NOT NULL | 입력 방식 (MANUAL/AUTO_API/AUTO_SMS) |

### (3) DD-03 : budgets (예산)

| 컬럼명          | 타입          | 제약조건         | 설명                   |
|--------------|-------------|--------------|----------------------|
| budget_id    | VARCHAR(36) | PK, NOT NULL | 예산 UUID              |
| user_id      | VARCHAR(36) | FK → users   | 예산 소유자 ID            |
| category     | ENUM        | NOT NULL     | ALL / 개별 카테고리        |
| limit_amount | BIGINT      | NOT NULL     | 예산 한도 금액             |
| period       | ENUM        | NOT NULL     | DAILY/WEEKLY/MONTHLY |

### (4) DD-04 : challenge\_participations (챌린지 참여)

| 컬럼명              | 타입          | 제약조건            | 설명                       |
|------------------|-------------|-----------------|--------------------------|
| participation_id | VARCHAR(36) | PK, NOT NULL    | 참여 UUID                  |
| user_id          | VARCHAR(36) | FK → users      | 참여자 ID                   |
| challenge_id     | VARCHAR(36) | FK → challenges | 챌린지 ID                   |
| start_date       | DATE        | NOT NULL        | 챌린지 시작일                  |
| end_date         | DATE        | NOT NULL        | 챌린지 종료일                  |
| result           | ENUM        | NULL 허용         | IN_PROGRESS/SUCCESS/FAIL |
| points_awarded   | INT         | DEFAULT 0       | 지급된 포인트                  |

## 6. 구현 기술 설계

| 구분           | 기술/도구             | 버전/세부 기술                                           | 선택 이유                            |
|--------------|-------------------|----------------------------------------------------|----------------------------------|
| Frontend     | React Native      | TypeScript, Redux/Zustand, Axios, React Navigation | Android/iOS 동시 개발. 1인 개발 효율 극대화. |
| Backend      | Node.js (Express) | JavaScript, JWT, Sequelize ORM, Joi (유효성 검증)       | 빠른 I/O 처리, 풍부한 생태계, 개발 속도.       |
| AI 서버        | Python / Flask    | scikit-learn, Pandas, NumPy, IsolationForest 모델    | ML 라이브러리 풍부. REST API 서빙 용이.     |
| DB (Primary) | MySQL             | InnoDB, 트랜잭션 처리, 인덱스 최적화                           | 관계형 데이터 처리. 트랜잭션 보장 필수.          |
| DB (Cache)   | Redis             | 세션 토큰, 리포트 캐싱, Rate Limiting                       | Refresh Token 관리 및 API 응답 속도 개선. |
| 푸시 알림        | FCM               | Firebase Cloud Messaging SDK                       | Android/iOS 통합 푸시 알림 무료 제공.      |
| 인증           | JWT + OAuth2.0    | Access Token(1h) + Refresh Token(30d)              | Stateless 인증. 소셜 로그인 통합 용이.      |
| 인프라          | AWS EC2           | t2.micro (프리티어), GitHub Actions CI/CD              | 비용 최소화. 자동 배포 파이프라인 구성.          |
| 디자인          | Figma             | 컴포넌트 기반 디자인 시스템                                    | UI/UX 프로토타입 및 디자인 핸드오프.          |
| 테스트          | Jest / Postman    | 단위 테스트, API 통합 테스트                                 | Frontend/Backend 자동화 테스트.        |
| 형상관리         | Git / GitHub      | Branching 전략: main / develop / feature/*           | 소스 버전 관리 및 CI/CD 연동.             |

## 7. 요구사항 추적표

| 요구사항 ID     | 요구사항명           | 설계 컴포넌트                 | 비고                                                        |
|-------------|-----------------|-------------------------|-----------------------------------------------------------|
| FR-001~006  | 사용자 인증 및 계정 관리  | SDD-P-001 (Auth)        | CM-001(UserAccount), CM-002(JwtTokenManager), M-001~006   |
| FR-010~016  | 소비 리포트 및 분석     | SDD-P-002 (Consumption) | CM-003(ConsumptionRecord), M-007~010                      |
| FR-020~024  | 예산 설정 및 알림      | SDD-P-002 (Consumption) | CM-004(BudgetManager), M-011~012, IF-002(FCM)             |
| FR-030~036  | 절약 챌린지          | SDD-P-003 (Challenge)   | CM-005(Challenge), M-013~014                              |
| FR-040~044  | 보상 및 인센티브       | SDD-P-003 (Challenge)   | CM-006(RewardSystem), M-015~017, IF-003(기프트콘)             |
| FR-050~053  | AI 기반 충동소비 탐지   | SDD-P-004 (AI)          | CM-007(ImpulseDetector), M-018~020, IF-002(FCM)           |
| FR-060~065  | 절약 커뮤니티         | SDD-P-005 (Community)   | CM-008(Post), M-021~023, IF-002(FCM)                      |
| NFR-010~015 | 보안 요구사항         | 전체 패키지 공통               | JWT(CM-002), bcrypt(CM-001), HTTPS(IF 전체), AES-256(DD-01) |
| NFR-001~005 | 성능 요구사항         | 전체 패키지 공통               | Redis 캐싱, DB 인덱스 최적화, AI 서버 분리 구조                         |
| IR-010      | 카드사 Open API 연동 | SDD-P-002, IF-001       | MVP에서는 수동 입력 우선, 자동 연동은 Sprint 3 적용                       |
| IR-012      | FCM 푸시 알림       | SDD-P-002~005, IF-002   | 예산/챌린지/AI/커뮤니티 알림 통합 관리                                   |

## 8. 부록

### 8.1 API 엔드포인트 요약

| 패키지         | HTTP Method | 엔드포인트                    | 설명               |
|-------------|-------------|--------------------------|------------------|
| Auth        | POST        | /v1/auth/register        | 회원가입 (M-001)     |
|             | POST        | /v1/auth/login           | 로그인 (M-002)      |
|             | POST        | /v1/auth/refresh         | 토큰 갱신            |
| Consumption | POST        | /v1/consumptions         | 소비 내역 입력 (M-007) |
|             | GET         | /v1/consumptions/weekly  | 주간 리포트 (M-009)   |
|             | GET         | /v1/consumptions/monthly | 월간 리포트 (M-010)   |
|             | POST        | /v1/budgets              | 예산 설정 (M-011)    |
| Challenge   | GET         | /v1/challenges           | 챌린지 목록 조회        |
|             | POST        | /v1/challenges/{id}/join | 챌린지 참여 (M-013)   |
| AI          | POST        | /ai/detect               | 충동소비 탐지 (M-018)  |
|             | GET         | /ai/tips/{userId}        | 절약 팁 제공 (M-019)  |
| Community   | POST        | /v1/posts                | 게시글 작성 (M-021)   |
|             | GET         | /v1/posts                | 게시글 목록 조회        |
|             | POST        | /v1/posts/{id}/like      | 게시글 좋아요 (M-022)  |

### 8.2 참고 문헌

| No. | 문헌명                          | 출처                                                                              | 비고             |
|-----|------------------------------|---------------------------------------------------------------------------------|----------------|
| [1] | IT@CookBook, 소프트웨어 공학 이론과 실제 | 한빛아카데미                                                                          | 교재             |
| [2] | React Native 공식 문서           | <a href="https://reactnative.dev/docs">https://reactnative.dev/docs</a>         | Frontend 개발 참조 |
| [3] | Node.js / Express 공식 문서      | <a href="https://nodejs.org/en/docs">https://nodejs.org/en/docs</a>             | Backend 개발 참조  |
| [4] | scikit-learn 공식 문서           | <a href="https://scikit-learn.org">https://scikit-learn.org</a>                 | AI 모델 개발 참조    |
| [5] | Firebase Cloud Messaging 문서  | <a href="https://firebase.google.com/docs">https://firebase.google.com/docs</a> | 푸시 알림 연동       |

---

|     |                   |                                                                                                             |          |
|-----|-------------------|-------------------------------------------------------------------------------------------------------------|----------|
| [6] | GitHub Repository | <a href="https://github.com/Niicka/Software-Engineering">https://github.com/Niicka/Software-Engineering</a> | 프로젝트 저장소 |
|-----|-------------------|-------------------------------------------------------------------------------------------------------------|----------|

해당사항 없음 (기타 부록)